private Node CreateTree(int[] in, int si, int ei) { // TODO Auto-generated method stub
if(si>ei) {
return null;
}
int mid = (si + ei) / 2;
Node nn = new Node(in[mid]);
nn.left = CreateTree(in, si, mid - 1);
nn.right = CreateTree(in, mid + 1, ei);
return nn;
}

public TreeNode insertIntoBST(TreeNode root, int val) { if(root==null) {
return new TreeNode(val);
}
if(root.val<val) {
root.right=insertIntoBST(root.right, val);
}
else {
root.left=insertIntoBST(root.left, val);
}
return root;

}

public TreeNode deleteNode(TreeNode root, int key) {

if(root.val<key) {
root.right=deleteNode(root.right, key);
}
else if(root.val>key) {
root.left=deleteNode(root.left, key);
}
else {
// 1 child
if(root.left==null) {
return root.right;
}
else if(root.right==null) {
return root.left;
}
}

}

public BstPair ValidBST(TreeNode root) {
if(root==null) {
return null;
}
BstPair lbp=ValidBST(root.left);
BstPair rbp=ValidBST(root.right);
BstPair sbp = new BstPair();
sbp.min=Math.min(root.val, Math.min(lbp.min, rbp.min)); sbp.max=Math.max(root.val, Math.max(lbp.max, rbp.max)); if(lbp.isbst && rbp.isbst && lbp.max<root.val && rbp.min>root.val) { sbp.isbst=true;
}
else {
sbp.isbst=false;
}
return sbp;
}

preorder = [3,9,20,15,7],
inorder = [9,3,15,20,7]

Lec-22

17/07/26 10:17 AM